

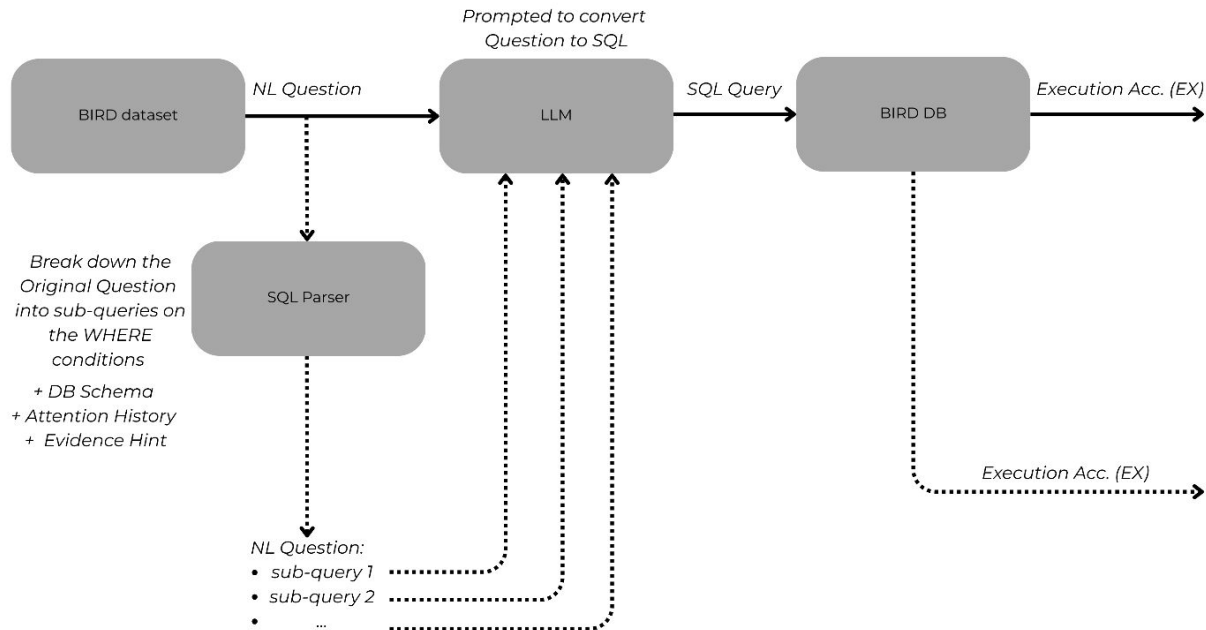
Detecting Hallucinations in Text-to-SQL with Progressive Queries (Update)

Hayden Moore (hmm5731@psu.edu)

May 9 2025

Agenda

- Review our pipeline/experiments so far
- Review of recent papers using sub-queries
- How we are clearly differentiating from these papers
- Example of a Hallucination from my original experiments
- Review restricting to SELECT, FROM, WHERE structure



Baseline performance on Mini-Dev Dataset

EX Evaluation

	SQLite	MySQL	PostgreSQL
mixtral-8x7b	21.60	13.60	12.40
llama3-8b-instruct	24.40	24.60	18.40
phi-3-medium-128k-instruct	30.60	25.00	21.60
gpt-35-turbo-instruct	33.60	31.20	26.60
gpt-35-turbo	38.00	36.00	27.40
llama3-70b-instruct	40.80	37.00	29.40
TA + gpt-35-turbo	41.60	-	-
TA + llama3-70b-instruct	42.80	-	-
gpt-4-turbo	45.80	41.00	36.00
gpt-4-32k	47.00	43.20	35.00
gpt-4	47.80	40.80	35.80
TA + gpt-4-turbo	58.00	-	-
TA + gpt-4o	63.00	-	-

Execution Acc. (EX)

GPT-4-Turbo: 38.40

Updates

2 new papers we have discovered:

- HLR-SQL: Human-Like Reasoning for Text-to-SQL:
<https://openreview.net/pdf?id=NZLm4vzXfm>
- CHASE-SQL: Multi-Path Reasoning and Preference Optimized Candidate Selection in Text-to-SQL: <https://arxiv.org/pdf/2410.01943>

Both reporting improved performance on BIRD benchmark using a similar approach where they break down the original NL questions into sub-questions

Both do not once mention Hallucinations:



A screenshot of a search interface. On the left, there is a search bar containing the text "Hallucination". To the right of the search bar are several search filters: a checked checkbox for "Highlight All", and unchecked checkboxes for "Match Case", "Match Diacritics", and "Whole Words". On the far right, the text "Phrase not found" is displayed.

HLR-SQL

1. Iterative, Human-Like Reasoning

The system mimics human problem-solving by incrementally building SQL queries through multiple steps. Instead of generating the entire query at once, it starts with simple sub-queries (e.g., `SELECT * FROM table`), tests them against the database, and iteratively refines or combines them into more complex statements. This reduces error propagation and allows mid-process corrections.

2. Autonomous Refinement Capabilities

HLR-SQL employs an LLM agent that autonomously decides between actions like generating sub-questions, creating/combining SQL fragments, or finalizing the query. This flexibility enables adaptive strategies based on intermediate results (e.g., adjusting join conditions after testing partial queries).

3. External Memory for Context Retention

The system maintains a memory of executed sub-queries and their results, providing contextual grounding for subsequent steps. This allows the LLM to reference previous work (e.g., reusing validated join conditions) and avoid redundant computations.

4. Spider-HJ Benchmark for Stress Testing

To validate performance gains, the authors created Spider-HJ—a modified version of Spider with 10× more average joins (5.64 vs. 0.54) and a maximum of 20 joins. HLR-SQL achieved a **9.51% accuracy improvement** over DAIL-SQL (the Spider leaderboard leader) on this benchmark, demonstrating scalability to real-world complexity

HLR-SQL (Spider-HJ)

The authors created **Spider-HJ** to address a critical gap in Text-to-SQL benchmarking: existing datasets like Spider lacked the **join complexity** seen in real-world enterprise databases, where queries often require 4+ joins

. Their intuition was twofold:

1. **Realism Gap in Benchmarks**

Real-world databases are highly normalized, but Spider averaged only **0.54 joins/query** and Bird **1.02**, so they created a benchmark where:

- Natural language questions remained unchanged
- SQL complexity increased **solely through added joins** (avg. 5.64 joins/query, max 20)

2. **Controlled Complexity Scaling**

By modifying Spider's schema to split columns into new normalized tables (e.g., splitting instructors(id, name, course) into instructors and courses tables)

HLR-SQL (Hallucinations)

There is no explicit discussion or mention of "hallucinations" in the context of large language models (LLMs) generating false or unsupported information. The term does not appear in the text, nor is there a section dedicated to analyzing or mitigating hallucinations as a phenomenon in LLM-based Text-to-SQL systems.

The paper does discuss error propagation and assumption errors in query generation, such as synonym mismatches or incorrect join keys, and describes how their iterative, human-like approach allows the system to detect and correct such errors through sub-query execution and refinement.

However, these are framed as reasoning or execution errors rather than as "hallucinations" in the LLM sense

CHASE-SQL

1. **Multi-Path Candidate Generation:**

CHASE-SQL generates diverse SQL query candidates using three complementary strategies: a divide-and-conquer method that decomposes complex questions into sub-queries, a query plan-based chain-of-thought approach that mirrors database execution steps, and an instance-aware synthetic example generation technique that creates tailored few-shot demonstrations for each test case.

2. **Iterative Query Fixing:**

After initial candidate generation, a query fixer module uses LLM-based self-reflection to iteratively correct syntactic or logical errors in the SQL queries, leveraging feedback from execution errors or empty results to improve candidate quality.

3. **Preference-Optimized Candidate Selection:**

A fine-tuned selection agent performs pairwise comparisons among the candidate queries, scoring and ranking them to robustly identify the most accurate SQL query for the given natural language input. This approach outperforms traditional consistency-based selection methods

4. **Comprehensive Value Retrieval and Contextualization:**

The framework begins by retrieving relevant database values and schema information using LLMs with few-shot prompts and locality-sensitive hashing, ensuring that candidate generation is grounded in the actual data context and robust to typos or semantic ambiguity in the input

CHASE-SQL (Hallucinations)

There is **no explicit discussion of hallucinations**-either the term itself or the phenomenon as commonly defined in the context of large language models (LLMs)-in the main text, error analysis, or methodological sections of the paper.

The paper focuses on improving Text-to-SQL performance through multi-path candidate generation, iterative query fixing, and a preference-optimized selection agent, but it does not address or mention hallucinations (i.e., the generation of plausible but incorrect or unsupported content) as a challenge or evaluation criterion.

The error analyses concentrate on issues such as schema linking errors, incorrect logic, SQL function errors, join errors, and ignoring evidence, but do not frame these as hallucinations or discuss mitigation strategies specifically targeting hallucination phenomena

Our Approach

Before our awareness to these two papers we approached this with a very similar idea of breaking down the queries into subqueries as this iterative process mimics more closely to human interactions with queries.

Our results from the BIRD-mini dataset resulted in very similar improvements by using these progressive sub-queries and using context retention, especially when looking at the “challenging” difficulty.

Our Results

Model:	Experiment #	Prompt Used	Simple EX	Moderate EX	Challenging EX	Total EX
GPT-4-Turbo	exp_1	1	56.76	29.60	13.73	34.40

Model:	Experiment #	Prompt Used	Simple EX	Moderate EX	Challenging EX	Total EX
GPT-4-Turbo	exp_2	1	57.43	34.00	14.71	37.00
GPT-4-Turbo (Partial Queries)	exp_12	1	55.41	34.40	17.65	37.20

Model:	Experiment #	Prompt Used	Simple EX	Moderate EX	Challenging EX	Total EX
GPT-4-Turbo	exp_11	4	56.76	32.00	15.69	36.00
GPT-4-Turbo (Partial Queries)	exp_10	4	54.05	36.80	19.61	38.40

Our Approach (Focusing on Hallucinations)

Given that we can match and even improve performance by using progressive queries it seems like it also lends itself more context/information into each query by looking at the progressive query performance.

We think by analyzing points where the model fails along the progressive path, we might be able to provide insights into underlying model limitations (where it might be more prone to hallucination) and what aspects of SQL structure present particular challenges for the LLM.

Our Approach (Focusing on Hallucinations)

```
▼ 38:
  question_id: 1524
  db_id: "debit_card_specializing"
  question: "'What is the country of the gas station where a transaction occurred on 2012/8/24?'"
  evidence: "'2012/8/24' can be represented by '2012-08-24'; spent 548.4 refers to price = '548.4' "
  SQL: "SELECT T2.Country FROM transactions_1k AS T1 INNER JOIN gasstations AS T2 ON T1.GasStationID = T2.GasStationID WHERE T1.Date = '2012-08-24'"
  difficulty: "simple"
  is_original: false

▼ 39:
  question_id: 1524
  db_id: "debit_card_specializing"
  question: "'What's the nationality of the customer who spent 548.4 in 2012/8/24?'"
  evidence: "'2012/8/24' can be represented by '2012-08-24'; spent 548.4 refers to price = '548.4' "
  SQL: "SELECT T2.Country FROM transactions_1k AS T1 INNER JOIN gasstations AS T2 ON T1.GasStationID = T2.GasStationID WHERE T1.Date = '2012-08-24' AND T1.Price = '548.4'"
  difficulty: "simple"
  is_original: true

"question_id": 1524,
"path": [
  {
    "question_id": 1524,
    "db_id": "debit_card_specializing",
    "query": "SELECT T2.Country FROM transactions_1k AS T1 INNER JOIN gasstations AS T2 ON T1.GasStationID = T2.GasStationID WHERE T1.Date = '2012-08-24'",
    "predicted_result": 1,
    "gt_result": 1,
    "correct": true,
    "error": null
  },
  {
    "question_id": 1524,
    "db_id": "debit_card_specializing",
    "query": "SELECT T2.Country FROM transactions_1k AS T1 INNER JOIN gasstations AS T2 ON T1.GasStationID = T2.GasStationID WHERE T1.Date = '2012-08-24' AND T1.Price = '548.4'",
    "predicted_result": 0,
    "gt_result": 1,
    "correct": false,
    "error": null
  }
]
},
```

Partial Question 1

What is the country of the gas station where a transaction occurred on 2012/8/24?

Ground Truth Query:

```
SELECT T2.Country FROM transactions_1k AS T1 INNER JOIN gasstations AS T2 ON T1.GasStationID = T2.GasStationID WHERE T1.Date = '2012-08-24'
```

Our Query:

```
SELECT DISTINCT customers.currency FROM transactions_1k JOIN customers ON transactions_1k.customerid = customers.customerid WHERE transactions_1k.date = '2012-08-24'",
```

Partial Question 2

What's the nationality of the customer who spent 548.4 in 2012/8/24?

Ground Truth Query:

```
SELECT T2.Country FROM transactions_1k AS T1 INNER JOIN gasstations AS T2 ON T1.GasStationID = T2.GasStationID WHERE T1.Date = '2012-08-24' AND T1.Price = '548.4'
```

Our Query:

```
SELECT DISTINCT customers.currency FROM transactions_1k JOIN customers ON transactions_1k.customerid = customers.customerid WHERE transactions_1k.date = '2012-08-24' AND transactions_1k.time = '16:25:00'",
```

```
▼ 30:
  question_id: 1514
  @b_16:      "debit_card_specializing"
  question:    "What kind of currency did the customer use in transactions on 2012/8/24?"
  evidence:    "'2012/8/24' can be represented by '2012-08-24'; "
  SQL:        "SELECT DISTINCT T3.Currency FROM transactions_1k AS T1 INNER JOIN gasstations AS T2 ON T1.GasStationID = T2.GasStationID INNER JOIN customers AS T3 ON T1.CustomerID = T3.CustomerID WHERE T1.Date = '2012-08-24'"
  difficulty:  "simple"
  is_original: false
▼ 31:
```

Next Steps

- Hallucination Metrics
- Look deeper into papers uses of context history (are they giving for context/query context?)
- Frequency analysis/bucketing